



Непрерывная разработка и интеграция CI/CD

...

Лекция 6

Волков Михаил Юрьевич

Тема 6

Конфигурирование и управление конвейерами сборки ПО

Конвейеры сборки ПО

Конвейер сборки — это набор параметров, управляющих процессом сборки. Параметры включают в себя такие вещи, как какая ветка системы управления версиями будет использована, настраиваемые сценарии и триггеры (плановая сборка или непрерывная интеграция / сборка после каждого коммита кода).

O Go Continuous Delivery

GoCD – проект с открытым исходным кодом, написанный на языке Java, а компоненты его веб-интерфейса созданы с использованием фреймворка Ruby on Rails.

Можно скачать GoCD для Windows, OSX, Debian и дистрибутивов Linux на базе RPM и Solaris. ThoughtWorks предоставляет коммерческую поддержку.

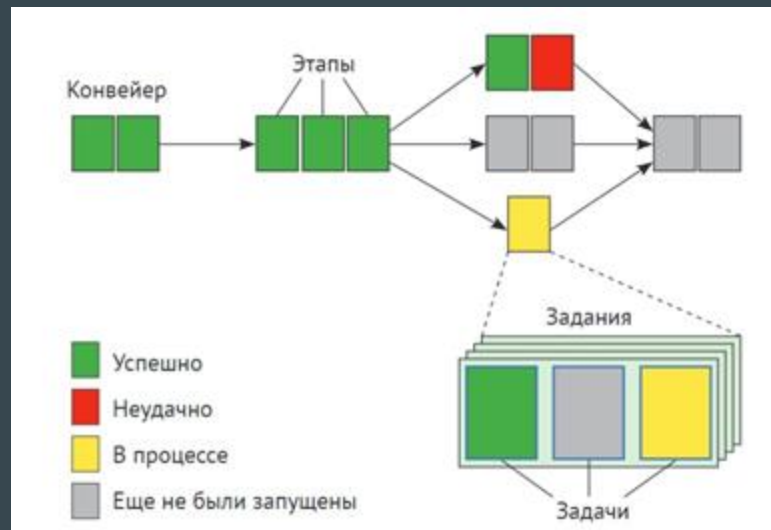
Created by
ThoughtWorks®



Устройство конвейера

Каждая сборка, развертывание или тестовое задание, выполняемое GoCD, должно быть частью конвейера. Конвейер состоит из одного или нескольких линейно расположенных этапов. Внутри этапа одно или несколько заданий могут выполняться параллельно и распределяться индивидуально по агентам.

Конвейеры могут запускать другие конвейеры, позволяя сформировать ациклический ориентированный граф конвейеров.



Соответствие заданий агентам

Когда агент простаивает, он опрашивает сервер на предмет наличия работы. Если на сервере есть задания для запуска, используется два критерия, чтобы определить, подходит ли агент для выполнения задания: среды и ресурсы.

Среды GoCD

GoCD позволяет запускать агенты в определенных средах.

Можно запускать агента Go при каждом тестировании и на каждом производственном компьютере и сопоставлять конвейеры с агентскими средами, чтобы гарантировать, что этап установки выполняется на правильном компьютере в правильной среде.

Материалы в GoCD

Материал в GoCD служит двум целям: запуск конвейера и предоставление файлов, с которыми могут работать задачи в конвейере.

Существуют плагины для материалов, предназначенные для различных систем контроля версий, таких как Subversion (svn) и mercurial, а также плагины для обработки репозитория пакетов Debian и RPM в качестве материалов.



Создание и настройка заданий и конвейеров с использованием YAML

YAML – это удобочитаемый язык сериализации данных, который обычно используется для файлов конфигурации. Использование файла YAML (файл с расширением .yaml) открывает возможности для хранения задания или конфигурации конвейера в любом из репозиториях Git проекта.



Создание и настройка заданий и конвейеров с использованием YAML

```
job:
  name: MyFirstYAMLJob
  vm-template: Basic Build VM Template
  git:
    - url: "https://mydevcsinstance-
mydomain.developer.ocp.oraclecloud.com/mydevcsinstance-
mydomain/s/mydevcsinstance-mydomain my-
project_902/scm/employee.git"
    branch: master
    repo-name: origin
  steps:
    - shell:
        script: "echo Build Number: $BUILD_NUMBER"
    - maven:
        goals: clean install
        pom-file: "employees-app/pom.xml"
  after:
    - artifacts:
        include: "employees-app/target/*"
  settings:
    - discard-old:
        days-to-keep-build: 5
        builds-to-keep: 10
        days-to-keep-artifacts: 5
        artifacts-to-keep: 10
```

Система сборки постоянно отслеживает репозитории Git и при обнаружении файла YAML создает или обновляет задание или конвейер с указанной конфигурацией.

Создание и настройка заданий и конвейеров с использованием YAML

Все файлы YAML должны находиться в каталоге `.ci-build` в корневом каталоге главной ветки любого размещенного репозитория Git. Файлы YAML в других ветках игнорируются.

Система сборки постоянно отслеживает Git-репозитории проекта. Когда он обнаруживает обновление файла с расширением `.yaml` в каталоге `.ci-build` главной ветки репозитория Git, он сканирует файл, чтобы определить, является ли он заданием или конвейером, и создает или обновляет соответствующее задание или конвейер.

С какой целью создан YAML?

YAML создан для упрощения сериализации сложных структур данных.

Основными целями языка являются:

- Легкая читаемость человеком.
- Переносимость между языками программирования.
- Отвечать встроенным структурам данных динамических языков.
- Применять последовательную модель для поддержки общих инструментов.
- Поддерживать обработку в один проход.
- Выразительность и расширяемость.
- Легкость в реализации и использовании.

Преимущества YAML

1. Удобство восприятия и освоения человеком;
2. Независимость от языка программирования;
3. Однозначность;
4. Пригодность для контроля версий;
5. Строгость синтаксиса;
6. Скорость обработки;
7. Безопасность реализации.

Примеры использования YAML

- Файлы конфигурации (например, для Docker, Ansible, Docker Compose, Google App Engine, Ruby on Rails и Symfony);
- Совместное использование данных программами на различных языках;
- Файлы журналов;
- Отладка сложных структур данных;
- Обмен сообщениями между процессами;
- Хранение объектов.

Характерные особенности YAML

- Удобство для чтения
- Несколько документов в одном файле
- Строки необязательно заключать в кавычки
- Возможность явного указания типов
- Ссылки



```
package.json  index.txt  config.yml  .circleci/  .github/  .vscode/  .yml

.circleci > {} config.yml > {} workflows

You, 4 months ago | 1 author (You)

1  version: 2.1
2
3  jobs:
4    deploy:
5      docker:
6        - image: circleci/push@0.11.0
7      steps:
8        - checkout
9        - run:
10          name: Installing dependencies
11          command: npm install
12        # - run: npm run build
13        - run:
14          name: Deploying app to prod
15          command: npm run build -- --mode=production
```

Синтаксис YAML

YAML поддерживает кодировки UTF-8 и UTF-16, поэтому ключи и значения могут содержать не только латиницу. В YAML учитывается регистр символов. Для форматирования структур в YAML используются отступы. Они состоят из пробелов, а табуляция не используется. Комментарии начинаются с символа «#», от которого он отделяется пробелом.

В блочном формате элементы списка начинаются с дефиса и пробела.

```
1. --- # Список: блочный формат
2. - Юпитер
3. - Сатурн
4. - Уран
5. - Нептун
```

Синтаксис YAML

Во встроенном формате элементы списка разделяются запятыми, за которыми следуют обязательные пробелы. Список заключается в квадратные скобки, как в JSON.

```
1. --- # Список: в одну строку
2.   [Юпитер, Сатурн, Уран, Нептун]
```

Синтаксис YAML

Словари представлены в виде пар ключей и значений. Значение отделяется от ключа двоеточием и пробелом. Пробел здесь обязателен. В формате JSON словари заключаются в фигурные скобки.

```
1. ---
2. # Словарь: блочный формат
3. Газовые гиганты:
4.   ледяные:
5.     - Уран
6.     - Нептун
7.   не ледяные:
8.     - Юпитер
9.     - Сатурн
10.
11. # Словарь: в одну строку
12. Газовые гиганты: {ледяные: [Уран, Нептун], не ледяные: [Юпитер, Сатурн]}
```

Синтаксис YAML

Документы в одном файле разделяются тремя дефисами.

```
1. ---
2. планета:
3.     название: Земля
4.     спутники: [Луна]
5. ---
6. планета:
7.     название: Марс
8.     спутники: [Фобос, Деймос]
```

Синтаксис YAML

Чтобы обозначить конец файла без начала нового, используются три точки.

Строки в YAML не обязательно заключать в кавычки. Строки без кавычек могут содержать кавычки. Одинарные кавычки используются, когда не требуется экранировать символы. Если в такую строку нужно вставить одинарную кавычку, она экранируется также одинарной кавычкой. Строки в двойных кавычках могут содержать экранированные символы в стиле языка C.

```
1. no_quotes: This string is not in quotes but it can contain ' and ".
2. single_quotes: 'This is a string in single quotes. It contains both single (')
   and double (") quotes.'
3. double_quotes: "This is a string in double quotes.\nIt contains both single (')
   and double (\") quotes\nand a new line characters."
```

Синтаксис YAML

Чтобы явно указать тип значения, используются теги, обозначаемые двумя восклицательными знаками.

```
1. ---
2. not-date: !!str 2022-01-12
```

Синтаксис YAML

Многие реализации YAML поддерживают пользовательские типы для сериализации. Это локальные типы. Они обозначаются одним восклицательным знаком и определяются в приложении с помощью библиотеки для парсинга YAML.

```
1. ---  
2. myCat: !Cat { name: Tom, color: red }
```

Синтаксис YAML

Ссылка обозначается амперсандом, а ее значение подставляется с использованием звездочки.

```
1. ---
2. references:
3.   val1: &ref This will be repeated
4.   val2: *ref
```

Спасибо за внимание !

